Argument Lab

Abstract reasoning. Strategic reporting. Reproducible evidence.

5,000	512	3
random instances	exhaustive graph fixtures	strategic agents

B.Tech project report | 8 November 2026

Reference: Gerhard Weiss (ed.), Multiagent Systems, 2nd edition, Chapter 5.

Submission bundle: source code, README, exact textbook fixtures, test evidence, experiment data, and a recorded browser demonstration.

1. Objective and scope

This project implements a finite Dung argumentation framework and a grounded direct argumentation mechanism. Its purpose is to compute acceptance under multiple semantics, explain individual calculations, and investigate whether agents can improve their utility by hiding arguments. A local browser application makes the algorithms available for graph construction, textbook demonstrations, and reproducible experiments.

The submission includes Python source, an HTML/CSS/JavaScript interface, automated unit and API tests, desktop/mobile browser tests, raw experimental evidence, this report, a README, and a recorded demonstration. The engine is exact for the supported graph sizes. It deliberately uses exhaustive enumeration so that the results are inspectable and suitable for an undergraduate project.

The reference is Gerhard Weiss (editor), Multiagent Systems, second edition, MIT Press, 2013, Chapter 5. The supplied textbook's printed pages 181-185 describe abstract semantics, and pages 199-201 describe the strategic mechanism and Theorems 5.3-5.4. ASCII names a1, a2, etc. represent the book's alpha-indexed arguments. The pasted assignment omitted its probability values; p = 0.1, 0.3, 0.5, 0.7, 0.9 are explicit provisional experiment choices.

2. Mathematical model

An argumentation framework AF = (A, R) consists of a finite set A and a directed attack relation R contained in A x A. The implementation permits self-attacks, asymmetric attacks, mutual attacks, and arbitrary directed cycles. An attack with an unknown endpoint is invalid.

For S contained in A, let S+ be the arguments attacked by S. S is conflict-free exactly when S and S+ are disjoint. S defends a precisely when every attacker of a is in S+. An unattacked argument is therefore defended even by the empty set. The characteristic function F(S) returns every defended argument, independently of whether that argument belongs to S.

S is admissible when it is conflict-free and S is contained in F(S). S is complete when it is conflict-free and S = F(S). The grounded extension is the least fixed point obtained by iterating F from the empty set. Preferred extensions are inclusion-maximal admissible sets, equivalently inclusion-maximal complete extensions. A stable extension is conflict-free and attacks every argument outside itself. A semi-stable extension is complete and has an inclusion-maximal range S union S+.

The distinction between maximal and maximum cardinality matters: the algorithms compare sets by proper inclusion. They do not discard an incomparable extension merely because another one has more elements.

Legal labellings satisfy both textbook equivalences: an argument is in iff all attackers are out, and out iff at least one attacker is in. Otherwise it is undecided. Every complete extension generates one such labelling: in = S, out = S+, undec = A minus (S union S+). This gives a bijection, avoiding a production enumeration of all 3^|A| candidate labellings.

Acceptance status depends on the selected semantics. An argument is skeptical when present in all extensions, credulous when present in at least one, and rejected when present in none. The interface displays skeptical before credulous to make its categories exclusive. If no stable extension exists, the application reports undefined acceptance instead of applying vacuous universal quantification.

3. Strategic mechanism and theorem checks

Arguments are partitioned into true ownership sets A_i. An agent may reveal any subset of its own set, including the empty set, but cannot invent or reveal another agent's arguments. Given reports, the mechanism induces the framework on their union and returns its grounded extension. Utility is the number of accepted arguments belonging to the agent's true ownership set, not the total accepted count.

The best-response algorithm fixes all other reports, enumerates every subset of the selected agent's ownership, evaluates the outcome, and returns every report attaining the maximum utility. Ties matter because truthful reporting need only be a weak best response.

Theorem 5.3 is checked by enumerating every agent, every combination of other-agent reports, and every possible own report. Full revelation is compared with each alternative. A strict improvement disproves strategyproofness and returns the agent, other reports, hidden/revealed arguments, both outcomes, and both utilities. If no strict improvement exists, truthfulness is weakly dominant. The check does not restrict other agents to truthful reports.

Theorem 5.4 is evaluated separately. It forbids internal direct or indirect defeat within an agent's arguments, where indirect defeat is described by an odd-length directed path in the full graph. The implementation explicitly adopts directed-walk parity reachability, permitting repeated vertices. From each source, breadth-first search explores states (vertex, parity), toggling parity on each attack. It visits at most 2|A| states per source, so cycles terminate safely.

The default includes same-endpoint odd cycles, since the book's quantified pair does not explicitly require distinct arguments. A distinct-endpoint option exists for comparison. Under a strict simple-path interpretation, parity-walk checking may reject additional graphs; results therefore record this conservative convention and never equate a failed condition with non-strategyproofness. The exhaustive Theorem 5.3 decision remains authoritative for measured strategyproofness.

For a partition into m agents and n arguments, a complete truthfulness audit considers m times 2^n agent/report/profile comparisons. Grounded outcomes are cached by the revealed union, so at most 2^n distinct induced outcomes are computed. The interface caps graphs at 12 arguments; experiments use 6-8. Cycle parity checking costs O(n(n+|R|)) over all sources. These are small-instance exact algorithms, not claims of large-scale performance.

4. Software architecture and interaction

The architecture separates four responsibilities. The pure Python engine stores immutable argument/attack sets and implements Dung semantics. The strategic layer validates ownership and reports, evaluates grounded outcomes, and returns audit witnesses. The experiment module creates seeded random instances and persists full evidence. A standard-library HTTP server provides JSON routes and serves the static browser interface over loopback.

The application adapter is further split into request validation, use-case services, synchronized experiment-job management and HTTP transport under argumentation/webapp. server.py is a small CLI launcher. Each server instance owns its job manager; immutable experiment requests and copied result snapshots prevent state leakage. Boundary validation rejects malformed JSON shapes, failures are logged consistently, and a health endpoint supports readiness checks. GitHub Actions defines cross-platform Python verification and Linux desktop/mobile browser testing.

The browser interface has four views. Framework studio offers examples, editable arguments/attacks, a directed SVG graph, fixed-point iteration, extension cards, set checks, acceptance status, and legal labellings. Strategic playground edits ownership and revealed reports and displays utility, best responses, and both theorem checks. Experiments runs background jobs, displays progress and aggregate fractions, downloads raw JSON, and reopens saved instances in the studio. Project guide explains definitions, corrections, and a live presentation sequence.

Graph nodes are keyboard-operable buttons as well as pointer targets. Status is represented by text in addition to color. Errors are displayed as alerts; job progress is announced as status. The layout supports desktop and mobile widths. Import validates the framework before replacing editor state, while export preserves ownership. No automatic graph persistence is implied: users export JSON to keep edits.

The application has no third-party Python runtime dependency, authentication service, external API, or internet requirement. It binds to 127.0.0.1 and is designed for a trusted local workstation. JSON-only write routes reject cross-origin browser requests. Payload size, graph size, and browser experiment counts are bounded. This is a coursework application rather than a public multi-user service.

5. Textbook validation and errata

Figure 5.1 and Examples 5.5-5.6: attacks are a2->a1, a4->a1, a3->a2, a5->a4. The sets empty, {a3}, {a5}, {a3,a5}, and {a1,a3,a5} are admissible. The set {a3,a5} defends a1 and is incomplete. Grounded iteration is empty -> {a3,a5} -> {a1,a3,a5}. The final set is the unique complete, preferred, stable, and semi-stable extension.

Figure 5.2 and Examples 5.7-5.8: a1 and a2 attack each other; a3 is unattacked and isolated. Complete extensions are {a3}, {a1,a3}, and {a2,a3}. Grounded is {a3}; preferred and stable extensions are {a1,a3} and {a2,a3}. There are three legal labellings. Under preferred semantics, a3 is skeptical and a1/a2 are credulous.

Example 5.10: x owns {a1,a4,a5}, y owns {a2}, and z owns {a3}; attacks are a1->a2, a2->a3, a3->a4, a3->a5. Full reports produce {a1,a3} and utilities (1,0,1). If x hides a1, the outcome becomes {a2,a4,a5}, giving x utility 2. This is a strict profitable deviation. The book's utility sentence swaps y/z ownership; the implementation uses the corrected values.

Example 5.11 adds a6, owned by x, and a6->a3. Full reports produce {a1,a4,a5,a6}, giving x utility 4. The exhaustive audit verifies truthfulness against every report profile, not only this full-report profile. The odd-walk condition nevertheless fails through a1->a2->a3->a4. The argument cancelling the negative effect is a6, correcting the book's reference to a5.

Additional fixtures cover a defence chain, mutual attack, an odd cycle with no stable extension, self-attack, an empty framework, empty agents, and harmless internal conflict. Abstract-framework tests do not attempt to implement the separate structured-argument or dialogue-game mechanisms elsewhere in the chapter.

6. Experimental method

The submitted experiment samples 1,000 instances for each of five probabilities, totaling 5,000. It uses Python's seeded PRNG with seed 2026. Each instance samples n uniformly from {6,7,8}; each ordered non-self attack is included independently with probability p. Cycles are allowed. Each argument is independently assigned uniformly to one of three agents, with rejection until all agents are nonempty. Thus ownership is uniform over surjective assignments and not over size vectors. The random stream continues across probability groups in the stated order.

Every instance independently runs exhaustive strategyproofness and the structural condition. The condition is not used as a shortcut for the strategyproofness result. Each record retains the graph, ownership, flags, comparison count, odd-walk witness where present, and a profitable deviation where found. Files identify every exceptional instance rather than retaining only summary counts.

The default excludes self-attacks because the assignment does not specify their sampling policy. A command-line and browser option enable them. This policy, the provisional p values, ownership sampling, seed, and odd-walk convention are stored with results so an instructor can reproduce or change them.

7. Results and interpretation



Attack p	Instances	Strategyproof	Fraction	Condition holds	Holds but not SP	SP but condition fails
0.1	1000	837	0.837	276	0	561
0.3	1000	188	0.188	9	0	179
0.5	1000	24	0.024	0	0	24
0.7	1000	4	0.004	0	0	4
0.9	1000	1	0.001	0	0	1

Overall, 1,054 of 5,000 sampled instances were strategyproof. The measured fractions decline with attack density in this seeded sample. This is an empirical observation for the specified generator; it is not a proof

that the relationship is monotonic in all populations. Each fraction has denominator 1,000, so rare high-density cases should be interpreted cautiously. No inferential claim beyond this reproducible sample is required.

7.1 Condition holds but strategyproofness fails

There are zero such instances at every p. All 285 condition-satisfying instances are strategyproof. The complete list is the empty JSON array in results/condition_counterexamples.json. This supports the sufficient condition on the sampled population, without replacing its mathematical proof. A nonzero count would require inspecting witnesses, cycle conventions, and implementation before claiming a contradiction to the theorem.

7.2 Strategyproof although the condition fails

There are 769 such instances. All are listed with their full graphs and ownership in results/converse_examples.json. Representative cases at each density are p0.1-trial1, p0.3-trial10, p0.5-trial29, p0.7-trial144, and p0.9-trial46. The independent audit verifies all profiles and deviations for every saved instance and supplies per-agent details for these representatives.

The sparse case p0.1-trial1 has arguments {a,b,c,d,e,f}, attacks c->f, d->a, d->c, d->e, and ownership Agent1={a}, Agent2={d,e,f}, Agent3={b,c}. The condition fails because Agent2 owns both endpoints of d->e. Under full own revelation Agent2 always accepts d and f, obtaining utility 2: d blocks c, protecting f. No report can accept both d and e, so it cannot accept all three owned arguments. Hiding d can at best exchange d for e and never exceeds utility 2. Agent1 is a singleton owner and cannot improve by hiding its only argument. Agent3 cannot influence d by hiding b/c; full revelation weakly dominates its subsets. This explains strategyproofness without relying on the structural condition.

The dense representative p0.9-trial46 has ownership Agent1={b,c}, Agent2={a,d,e}, Agent3={f}. Within the first two ownership sets, attacks prevent acceptance of more than one own argument in any conflict-free outcome; Agent3 is a singleton. A direct parity witness still falsifies the sufficient condition. Exhaustive independent enumeration checks 16, 8, and 32 other-agent profiles respectively and finds maximum hiding gain 0 for every agent. Merely having an upper bound of one is not by itself a proof; the profile comparisons establish that no full-report utility zero can be improved to one by hiding.

For the remaining three representative cases, the maximum hiding gain is also 0 for each agent. Details include truthful utility ranges and profile counts in results/independent-audit.json. The textbook's Example 5.11 offers a different, interpretable reason for the same phenomenon: additional owned defence cancels indirect self-defeat. Internal odd defeat therefore does not imply a profitable utility tradeoff.

7.3 Independent audit

The saved results were audited by scripts/audit_results.py, which imports no production solver. It computes grounded extensions using integer-mask unattacked-node propagation, precomputes all induced outcomes, and independently compares every own report against truthful revelation across every other-agent profile. All 5,000 decisions agree. This is stronger evidence than reloading a saved strategyproofness flag in the interface.

8. Testing and demonstration

Python tests cover definitions, exact textbook expectations, cycles, self-attacks, empty frameworks/agents, tied best responses, invalid ownership and reports, HTTP routes, cross-origin rejection, and background experiment jobs. An independent oracle enumerates every total in/out/undec labelling for each of the 512 directed three-node graphs, checking complete, preferred, stable, semi-stable, and grounded outputs. A separate propagation oracle checks strategyproofness on that entire graph space.

Playwright exercises the actual browser interface at desktop and mobile viewports: graph editing, keyboard node toggles, textbook sets, multiple labellings, acceptance changes, hiding incentives, Example 5.11, error handling, import/export, empty/self-attacking graphs, background experiments, downloads, saved-instance inspection, and responsive layout. Final local verification passed all 33 Python tests and all 24 browser tests (12 scenarios at each viewport), with no failures or skipped tests. Machine-readable results and an HTML report record the executed suites; see TESTING.md for final evidence locations. The expanded Python suite verifies malformed input, readiness, job failures and recovery, concurrency, bounded retention, immutable parameters and isolation between server instances.

The demonstration is a genuine recorded browser walkthrough, with on-screen explanations and no audio. DEMONSTRATION.md provides a 5-7 minute live presentation script and viva questions. Screenshots document the desktop studio, mobile studio, profitable hiding, and experiment dashboard.

9. Limitations and future work

The exact algorithms are exponential and intentionally bounded. The random experiment uses one seed and a finite sample; further seeds, confidence intervals, and additional p values would improve statistical analysis. The p values remain provisional until the instructor's missing values are supplied. Directed-walk parity is a documented conservative cycle convention and should be compared with an explicitly required simple-path convention if the instructor distinguishes them.

The UI supports local JSON persistence, not collaborative storage or user accounts. Its HTTP server is suitable for local demonstration, not internet deployment. Future extensions could add SAT/ASP-based semantics, larger-scale sampling, graph layout algorithms, and structured arguments while retaining independent correctness checks.

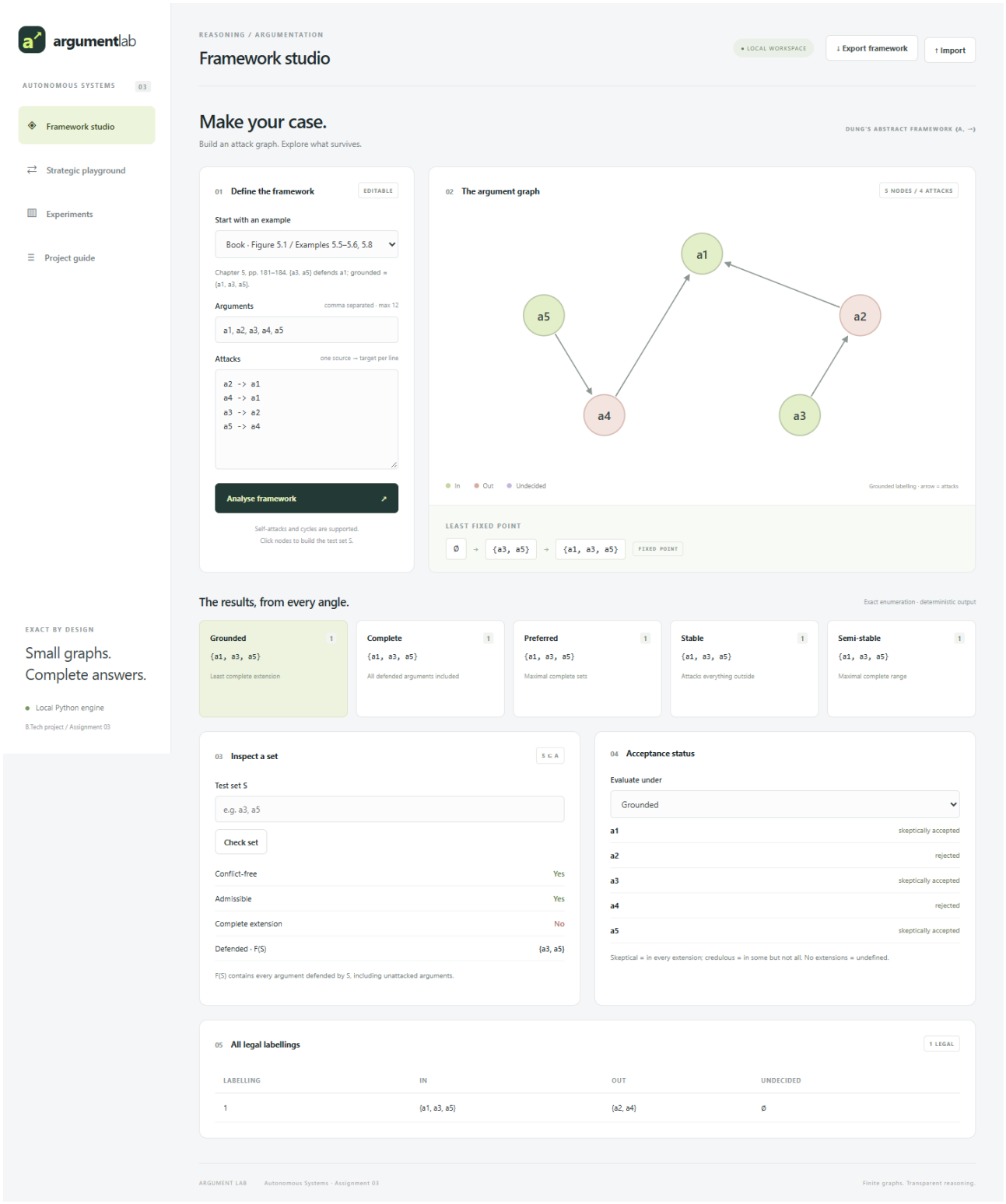
References

[1] Weiss, G. (ed.). Multiagent Systems. Second edition. MIT Press, 2013. Chapter 5, printed pp. 181-185 and 199-201. User-supplied book/multiagent-systems_compress.pdf.

[2] Rahwan, I. and Larson, K. Mechanism Design for Abstract Argumentation. AAMAS, 2008. https://cs.uwaterloo.ca/~klarson/papers/RahwanLarsonAAMAS2008.pdf. Related primary research; textbook numbering governs this assignment.

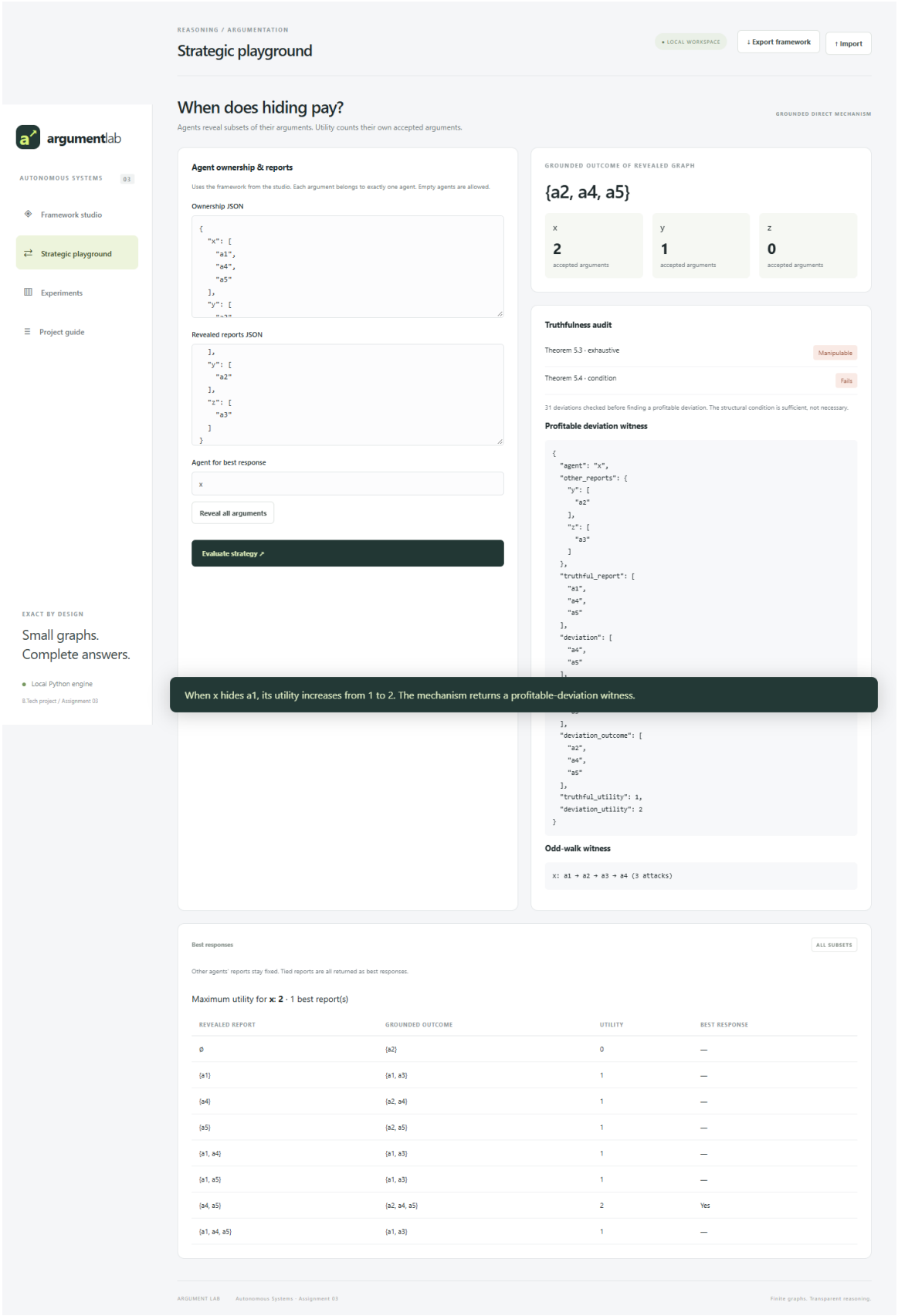
[3] Reproducible project evidence: results/experiments.json, results/summary.csv, results/converse_examples.json, and results/independent-audit.json. Generation seed 2026; executed on 8 November 2026.

Framework studio - desktop



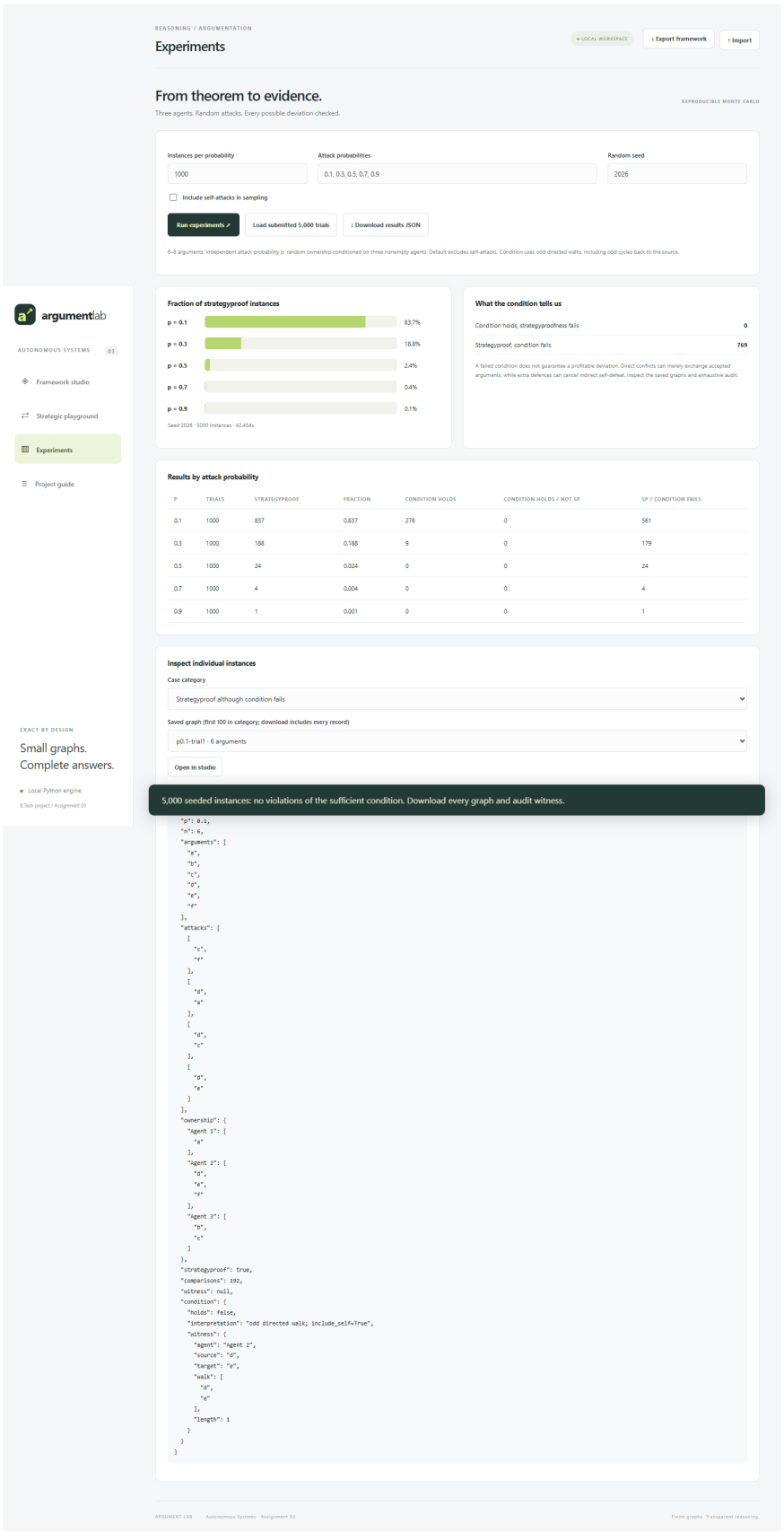
Screenshot captured from the running application by automated browser verification.

Profitable hiding - textbook Example 5.10



Screenshot captured from the running application by automated browser verification.

Experiment dashboard - submitted trials



Screenshot captured from the running application by automated browser verification.